Day 9~14 구현 지도 (비전공자용)

> 목적: 구현하다가 "아, 이게 그때 그 개념이다!"가 되도록 미리 지도를 그려둔다. 이 문서는 **개념 + 왜 + 어떻게 + 과정**을 담은 참고서다. (실제 코드는 세션 때 네 손으로 씀) 네 프로젝트 스택: **Gemini (google-genai)** / `build_prompt()` → `generate_feedback()` 구조. 표기: 🖐 = 네가 직접 칠 학습 대상 / 🧱 = 그냥 깔아주는 보일러플레이트

📖 이 문서 읽는 법 (중요 — 처음 읽을 때)

이 문서는 **두 가지 용도**로 쓰인다. 읽는 시점에 따라 보는 곳이 달라:

① **구현 전 "미리 이해"용 (지금 너)** → **글로 된 부분만 읽어.** 각 Day의 뭘 하나 / 개념 / 왜 하나 / 과정, 그리고 🔑 표시. → **회색 코드박스는 지금 다 이해하려 하지 마.** 넘겨도 돼. "이런 게 나오는구나" 정도만. → 코드박스 안의 어려운 문법(반복문, numpy 등)은 그때그때 세션에서 손으로 치며 배운다.

② **구현 중 "옆에 두고 참고"용 (나중)** → 그때 코드박스를 보며 골격을 따라 친다.

즉 **지금은 "무엇을·왜"만 이해하면 충분.** "어떻게(코드)"는 몰라도 정상이야. 코드가 안 읽혀도 겁먹지 마 — 원래 이 순서(개념 먼저, 코드 나중)로 배우는 거야.

---

0. 전체 그림 (이 한 장만 기억하면 됨)

지금 네 앱은 이렇게 돌아가:

```
아이정보 + 수업기록  →  build_prompt()  →  generate_feedback()  →  피드백 글
                        (주문서 작성)        (Gemini가 씀)
```

RAG를 붙이면 **주문서 작성 전에 "우리 학원 노하우 조각"을 찾아서 끼워넣는** 단계가 추가돼:

```
                      ┌─ [질문] → 검색 → 닮은 조각 몇 개 꺼내옴 ─┐
                      │            (Day 9)                      │
아이정보+수업기록 ────┘──→ build_prompt(...조각 끼움...) ──────┘──→ generate_feedback()
→ 피드백
                          (Day 10에서 조각 자리 추가)          (그대로)
```

그리고 마지막에 강사가 버튼으로 쓸 **화면**을 붙임:

```
[Streamlit 화면] → 우리 API 호출 → 위 파이프라인 → 결과를 화면에 표시
   (Day 12~14)
```

Day가 어디에 해당하나

Day	하는 일	한 줄 요약
9	임베딩 + 저장 + 검색	조각을 숫자로 바꿔 저장하고, 질문과 닮은 조각 찾기
10	RAG 연결	찾아온 조각을 `build_prompt`에 끼워넣기
11	다듬기·검증	"조각이 제대로 걸렸나" 눈으로 확인
12	Streamlit 기본	화면 요소(입력칸·버튼) 만들기
13	화면 ↔ API 연결	버튼 누르면 우리 API가 돌게 잇기
14	마감	다듬고 마무리

0.5 전체 구현 과정 (처음부터 끝까지 한 흐름)

> "지금 어디 있고 → 다음 뭘 하는지"를 한 줄기로 본다. 오른쪽은 **왜/뭘 하는지** 설명. ✅ = 이미 함 / 🔴 = 다음 차례

```
[준비] 조각 문서 쓰기 (rag_docs)                    ✅ Day 8 완료
   │
   ▼
① 파일 → 조각 리스트로 자르기 (chunking)            ✅ 방금 함 (kids.py)
   │
   ▼
② embed 함수 만들기: 조각 1개 → 벡터                🔴 여기부터 Day 9
   │
   ▼
③ 조각 전부 embed해서 db에 저장 (조각글+벡터 짝)
   │
   ▼
④ cosine 함수: 두 벡터 닮은 정도 점수
   │
   ▼
⑤ search 함수: 질문 → 닮은 조각 3개 꺼내기
   │
   ▼
⑥ search 테스트: 맞는 조각이 나오나 눈으로 확인
   │
   ▼
⑦ build_prompt에 [참고 노하우] 자리 추가            🔴 Day 10
   │
   ▼
⑧ router에서 search→context→build_prompt 잇기
   │
   ▼
⑨ 피드백 생성 테스트: 학원 노하우가 반영되나
   │
   ▼
⑩ rag_docs 초안을 실제 방식으로 교체                 🔴 Day 11
   │
   ▼
⑪ Streamlit 화면: 입력칸 + 버튼                     🔴 Day 12
```

```
    │
    ▼
⑫ requests로 우리 API 호출 연결                    🔴 Day 13
    │
    ▼
⑬ 결과를 화면에 표시
    │
    ▼
⑭ 전체 통 테스트 + README + 마감                   🔴 Day 14
    │
    ▼
[완성] 강사가 화면 버튼만으로 피드백 받기 🎉
```

옆 설명 (각 단계가 왜/뭐)

순서	무엇을 한다	왜 / 무슨 뜻
①	파일을 ## 기준으로 조각 리스트로	컴퓨터가 조각 단위로 다루게 하려고 (검색의 재료)
②	조각 하나를 벡터(숫자목록)로	글자는 못 비교 → 숫자로 바꿔야 "닮음"을 계산 가능
③	모든 조각을 벡터로 만들어 창고에 보관	질문 올 때 비교하려면 미리 다 숫자로 만들어둬야 함
④	두 벡터의 닮은 정도를 점수로	"이 조각이 질문과 얼마나 관련 있나"를 수치화
⑤	질문과 제일 닮은 조각 3개를 꺼냄	이게 RAG의 "찾아서(Retrieval)" 핵심 동작
⑥	넣어보고 결과 조각을 눈으로 검사	엉뚱한 게 걸리면 조각/질문을 고쳐야 하니까
⑦	프롬프트에 노하우 들어갈 빈자리 만듦	찾아온 조각을 Gemini에게 넘겨줄 통로
⑧	검색→합치기→프롬프트를 순서대로 연결	흩어진 함수들을 하나의 흐름으로 잇기
⑨	실제 피드백이 학원 방식으로 나오나 확인	RAG를 붙인 목적이 이뤄졌는지 최종 점검
⑩	임시 초안을 진짜 학원 노하우로 교체	AI가 모르는 "우리만의 것"이라야 가치 있음
⑪	강사가 쓸 입력칸·버튼 화면 만들기	코드/터미널 모르는 강사도 쓰게
⑫	버튼 → 우리 서버 호출 연결	화면과 서버는 남남 → 요청으로 대화시킴
⑬	받은 결과를 화면에 뿌리기	강사 눈에 피드백이 보이게
⑭	통 테스트·문서·마무리	처음~끝 한 번에 돌아가는지 확인하고 종료

> 💡 이 14단계가 남은 전부다. **지금 ①까지 왔고, ②(embed 함수)가 바로 다음.** 한 번에 다 보지 말고 "지금 ② 구나" 하고 그 칸만 보면 돼.

Day 9 — 임베딩 + 저장 + 검색 🔴 (제일 큰 고비)

> 이미 한 것: chunking (##로 조각내기). 이제 그 조각들을 "검색 가능"하게 만든다.

9-1. 이 단계에서 뭘 하나

1. 조각들을 **숫자(벡터)로 바꾼다** = 임베딩(embedding)
2. 그 숫자들을 **저장**해둔다
3. 질문이 오면 질문도 숫자로 바꿔서 **가장 닮은 조각을 찾아온다** = 검색(search)

9-2. 개념 ①: 임베딩 = "글을 숫자 목록으로 바꾸는 것"

컴퓨터는 "위축형", "칭찬" 같은 **글자의 뜻**을 모른다. 대신 글을 **숫자 목록**으로 바꿔놓으면, 숫자끼리 얼마나 가까운지로 "뜻이 비슷한지"를 계산할 수 있다.

```
"위축형 아이 칭찬법"   →  [0.12, -0.44, 0.98, ... ]   ← 숫자 수백 개짜리 목록 (이게 벡터)
"자신감 없는 애 칭찬"  →  [0.11, -0.41, 0.95, ... ]   ← 뜻이 비슷하면 숫자도 비슷하게 나옴
"줄넘기 준비물"        →  [-0.88, 0.02, 0.31, ... ]   ← 뜻이 다르면 숫자도 딴판
```

- **벡터(vector)** = 그냥 "숫자 목록"이라는 뜻. 무서운 말 아님.
- **왜 하나:** 컴퓨터가 "뜻이 비슷한 걸 찾게" 하려고. 글자로는 못 비교하니까 숫자로 바꾸는 것.
- 이 변환을 해주는 게 **임베딩 모델** (우리는 Gemini의 임베딩 기능을 부른다).

> 🔑 "아 이게 그 개념": chunk에서 배운 ②**독립 테스트**가 여기서 빛을 봐. 조각이 혼자서 뜻이 완결돼야, 숫자로 바꿨을 때도 뜻이 또렷하게 찍혀서 검색이 잘 된다.

9-3. 개념 ②: 유사도 = "두 숫자 목록이 얼마나 닮았나"

숫자로 바꿔놨으면, 두 벡터가 **얼마나 닮았는지** 점수를 낸다. 이 점수를 **유사도(similarity)**라 한다. 가장 많이 쓰는 방식이 **코사인 유사도** — "두 화살표가 같은 방향을 보나"를 재는 것 (1에 가까울수록 닮음).

- 이유까지 다 알 필요 없음. **"닮을수록 점수 높다"**만 알면 시작 가능.
- 검색 = 질문 벡터와 **모든 조각 벡터의 유사도를 재서, 점수 높은 순 몇 개를 꺼내는 것.**

9-4. 과정 (5단계)

```
① chunking       : 파일 → 조각 리스트          ← 이미 함 (kids.py)
② embed(조각들)  : 각 조각 → 벡터              ← Gemini embed 호출
③ 저장           : (조각글 + 벡터) 짝지어 보관  ← 리스트나 파일에
④ embed(질문)    : 질문도 벡터로               ← 같은 함수 재사용
⑤ search         : 질문벡터 ↔ 조각벡터들 유사도 → 상위 몇 개 조각 반환
```

9-5. 핵심 코드 골격 (패턴만 — 실제는 네가 채움)

깔 것 (🧱): 계산용 라이브러리 `numpy` (유사도 계산에 씀)

```
pip install numpy
```

(임베딩은 이미 있는 google-genai로 하니까 추가 설치 X)

② **임베딩 함수** 🖐 — 조각 하나를 벡터로 바꾸는 것. 우리 `generate_feedback`이 Gemini를 부른 것처럼, 이번엔 임베딩을 부른다:

```
# 개념 골격 (정확한 모델명·필드는 공식문서에서 확인해 채우기)
from google import genai
client = genai.Client(api_key=api_key)

def embed(text):
    result = client.models.embed_content(
        model="gemini-embedding-001",    # ← 임베딩 전용 모델 (생성 모델 gemini-2.5-flash와 다름!)
        contents=text,
    )
    return result.embeddings[0].values    # 숫자 목록(벡터)만 꺼내서 돌려줌
```

> 🔑 "아 이게 그 개념": `generate_feedback`이 "글 써줘"라고 Gemini 부른 거랑 똑같은 패턴. 이번엔 "이 글을 숫자로 바꿔줘"라고 부르는 것뿐. **client 부르기 → 결과에서 필요한 값 꺼내 return.**

③ **저장** 🖐 — 조각글과 벡터를 짝지어 보관:

```
chunks = text.split("##")                  # 조각 리스트 (Day 8~9)
db = []                                      # 우리만의 작은 창고
for c in chunks:
    db.append({"text": c, "vector": embed(c)})    # {조각글, 벡터} 한 세트씩 쌓기
```

⑤ **검색** 🖐 — 질문과 가장 닮은 조각 찾기 (numpy로 유사도):

```
import numpy as np

# 두 벡터가 얼마나 닮았는지 점수를 내주는 계산기 (1에 가까울수록 닮음)
def cosine(a, b):
    a, b = np.array(a), np.array(b)
    return np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b))

def search(question, top_k=3):
    q = embed(question)                    # 1) 질문을 벡터로 바꾼다
    scored = []                            # 2) (점수, 조각글) 담을 빈 리스트
    for item in db:                        # 3) 저장된 조각을 하나씩 꺼내며
        score = cosine(q, item["vector"])        # 질문과 얼마나 닮았나 점수 계산
        scored.append((score, item["text"]))     # (점수, 조각글) 짝을 리스트에 추가
    scored.sort(reverse=True)              # 4) 점수 높은 순으로 정렬
    top = scored[:top_k]                   # 5) 위에서 top_k개(기본 3개)만 자른다
    result = []                            # 6) 조각글만 골라 담을 리스트
    for score, text in top:
        result.append(text)                # 점수는 버리고 조각글만 담기
    return result                          # 닮은 조각 3개를 돌려줌
```

말로 풀면: ①질문을 숫자로 바꾸고 → ②③저장된 조각마다 "질문과 얼마나 닮았나" 점수를 매겨서 → ④점수순 줄 세우고 → ⑤위 3개만 → ⑥조각 글만 돌려준다.

- `np.dot`, `np.linalg.norm` = numpy가 주는 계산기 함수. **지금은 "닮은 정도를 숫자로 내주는 공식"으로만 이해하면 됨.** 안을 외울 필요 X.
- `db` = ③단계에서 만든 우리 창고 (조각글+벡터 세트들).

> 🔑 "아 이게 그 개념": `top_k=3` = "제일 닮은 3개만" (예전 `GET`에서 "최근 5개만" 자른 거랑 같은 발상).

9-6. Day 9 완료 기준

- `search("위축형 아이 칭찬")` 넣으면 → 위축형 칭찬 조각이 상위에 나오면 성공.
- 엉뚱한 조각이 나오면? 조각을 다시 손봐야 함 (독립·요약 테스트 위반일 확률 큼).

9-7. 공식문서 / 검색어

- Gemini 임베딩: **"Gemini API embeddings"** (ai.google.dev/gemini-api/docs/embeddings)
- 코사인 유사도: **"cosine similarity numpy"**
- ⚠️ 임베딩 모델명은 자주 바뀜 → 공식문서에서 현재 이름 확인하고 골격의 `model=` 채우기.

Day 10 — RAG를 피드백에 연결

10-1. 이 단계에서 뭘 하나

Day 9에서 만든 `search()`가 찾아온 조각을, `build_prompt()`**의 프롬프트 안에 끼워넣는다.** 그럼 Gemini가 "우리 학원 노하우"를 참고해서 피드백을 쓴다.

10-2. 왜 하나

지금 `build_prompt`는 아이 정보 + 수업기록만 넣는다. 여기에 **관련 노하우 조각**을 더 넣으면, Gemini가 일반론이 아니라 "이 학원 방식"으로 답한다. (RAG의 최종 목적)

10-3. 어떻게 — 과정

```
① 이 아이 성향으로 질문 만들기   예) "위축형 아이 칭찬법"
② search(질문) → 관련 조각 3개
③ 그 조각들을 하나의 글로 합치기
④ build_prompt에 [참고 노하우] 자리 새로 만들어 끼우기
```

10-4. 핵심 코드 골격 🖐

`prompt_service.py`의 `build_prompt`를 살짝 고친다:

```
def build_prompt(child, records, rag_context):    # ← 매개변수 하나 추가
    ...
    prompt = f"""[역할]
```

```
...
[최근 수업 기록]
{records_text}

[참고할 우리 학원 노하우]          ← 새 구역 추가
{rag_context}                       ← 검색해온 조각들이 여기 들어감

[작성 규칙]
...
"""
    return prompt
```

그리고 이 함수를 부르는 곳(feedbacks_router.py)에서 검색을 먼저 해서 넘긴다:

```
question = f"{child.personality} 아이 피드백"      # 아이 성향으로 질문 생성
context = "\n".join(search(question))              # 조각 3개를 한 덩어리 글로
prompt = build_prompt(child, records, context)     # 끼워서 주문서 완성
```

- "\n".join(리스트) = 리스트에 든 조각 3개를 **줄바꿈으로 이어 하나의 글**로 만드는 것. (split의 반대라고 보면 됨 — split은 자르고, join은 붙임)

> 🔑 "아 이게 그 개념": build_prompt에 records_text 끼운 거랑 **똑같은 방식.** 자리(구역) 하나 더 만들고, 거기에 검색 결과를 f-string으로 끼우는 것뿐.

10-5. 완료 기준

같은 아이라도 RAG 연결 전/후 피드백 톤이 "학원 노하우"에 맞게 달라지면 성공.

Day 11 — 다듬기 & 검증

11-1. 뭘 하나

"검색이 진짜 맞는 조각을 걸어오는지" 눈으로 확인하고 다듬는다.

11-2. 어떻게 (거창한 코드 아님, 확인 작업)

- 성향별로 질문 넣어보고 → search() 결과 조각이 말이 되는지 눈으로 검사.
- 이상하면 원인은 보통 둘: (a) 조각이 독립/요약 테스트 위반, (b) 질문 문구가 애매.
- 초안으로 채운 rag_docs를 **이때 네 학원 실제 방식으로 하나씩 교체** (Day 8에서 미뤄둔 것).

> 🔑 "아 이게 그 개념": Day 6 통합 테스트(아이 3명 넣고 톤 확인)랑 같은 결. "돌려보고 눈으로 확인 → 이상하면 원인 짚어 고치기."

Day 12 — Streamlit 기본 (화면 만들기)

12-1. Streamlit이 뭐고 왜 쓰나

Streamlit = 파이썬 몇 줄로 웹 화면을 만들어주는 도구. HTML/CSS 몰라도 됨. 강사가 터미널·코드가 아니라 **버튼과 입력칸**으로 우리 프로그램을 쓰게 하려고 붙인다.

12-2. 시작 최소 개념

- `st.title("제목")` → 큰 글씨 제목
- `st.text_input("라벨")` → 입력칸 (강사가 글 씀)
- `st.button("생성")` → 버튼 (누르면 아래 코드 실행)
- `st.write(값)` → 화면에 결과 출력

12-3. 핵심 골격 🖐

```
import streamlit as st

st.title("아이 피드백 생성기")
child_id = st.text_input("아이 ID를 입력하세요")
if st.button("피드백 생성"):        # 버튼 누르면 이 안이 실행됨
    st.write("여기에 결과 표시 예정")
```

깔 것 🧱: `pip install streamlit` / 실행: `streamlit run 파일.py`

12-4. 공식문서 / 검색어

- **"Streamlit get started"** (docs.streamlit.io)
- 유튜브: "Streamlit 기초 한국어"

Day 13 — 화면 ↔ 우리 API 연결

13-1. 뭘 하나

Day 12 버튼을 누르면 → **우리가 만든 FastAPI(피드백 생성 API)를 호출** → 결과를 화면에 표시.

13-2. 개념: 화면과 서버는 남남 → 요청으로 대화

Streamlit 화면과 FastAPI 서버는 **서로 다른 프로그램**이다. 화면이 서버에게 "이 아이 피드백 만들어줘"라고 **요청(request)**을 보내고, 서버가 결과를 **응답(response)**한다.

- 이 심부름꾼이 `requests` 라이브러리 (또는 httpx).

13-3. 핵심 골격 🖐

```
import requests

if st.button("피드백 생성"):
```

```
    res = requests.post(f"http://localhost:8000/feedbacks/generate",
                        json={"child_id": child_id})   # 우리 API 호출
    data = res.json()                                   # JSON 응답 받기
    st.write(data["parent_feedback"])                   # 화면에 표시
```

> 🔑 "아 이게 그 개념": 예전에 브라우저 `/docs`에서 손으로 눌러 API 부른 거를, 이제 **화면 버튼이 대신 눌러주는** 것. `json.loads`로 다뤘던 그 JSON이 여기 `res.json()`.

13-4. 검색어

- **"python requests post json"**
- **"streamlit call fastapi"**

Day 14 — 마감

14-1. 뭘 하나

- 화면 다듬기 (제목, 안내문구, 결과 예쁘게)
- 전체 흐름 한 번 통으로 테스트 (아이 등록 → 기록 → 버튼 → 피드백)
- README 마무리 (Day 7에서 밀린 것: API 명세 + ERD + 회고)
- 남은 rag_docs 초안 교체 마무리

14-2. 완료 기준

"강사가 화면만 보고 아이 피드백을 받을 수 있다" → 프로젝트 끝.

부록 A. 새로 깔 라이브러리 정리

Day	깔 것	용도
9	`numpy`	벡터 유사도 계산
12	`streamlit`	화면 만들기
13	`requests`	화면 → 서버 호출
(임베딩)	(추가 없음)	기존 google-genai 사용

부록 B. "아 이게 그 개념" 총정리

새로 보는 것	예전에 한 것
임베딩 함수 (client 부르고 결과 꺼냄)	`generate_feedback` (Gemini 호출 패턴)
`top_k=3` (닮은 3개)	`GET` 최근 5개 자르기

새로 보는 것	예전에 한 것
`build_prompt`에 rag_context 끼우기	`records_text` 끼운 것
`res.json()` 응답 다루기	`json.loads` 문자열→딕셔너리
Streamlit 검증	Day 6 통합 테스트

부록 C. 우선순위 조언

1. **Day 9가 남은 일의 1/3.** 임베딩·검색만 넘으면 확 쉬워짐. 2~3세션에 나눠서.
2. README는 머리 안 쓰는 일 → Day 9 사이 지칠 때 손 풀기용으로.
3. 초안 rag_docs 교체는 Day 11이나 14에 몰아서.